

Bit Flip Learning: Direct Selector Updates and Greedy MUX Networks

Facundo Gomez Prates

September 30, 2026 — theory and experiments

Abstract

This manuscript distinguishes two binary MUX-graph models in the BFL C++ library. The direct-state model uses the local update $S' = S \oplus (C \wedge U)$, with CHANGE given by an output mismatch and USED marking active dependencies. Count and partial-prefix routing select a state bank for this rule; the measured lookup topology gains route diversity without a held-out accuracy gain. The separate greedy model uses a signal-controlled selector XOR a learned bit and retains a proposed bit flip only when it reduces global training Hamming error. In a matched 20-split comparison, validation-selected greedy topologies reach mean held-out accuracies of .762, .733, and .608 on OR, majority, and XOR tasks; a small neural network reaches 1.000 on each. These observations are specific to the stated architectures and protocol, and neither rule has a generalization guarantee.

Keywords: Bit Flip Learning, multiplexers, binary state, discrete learning, active paths.

1 Motivation and scope

A multiplexer selects one of two signals using a binary selector. The library contains two distinct ways to learn selector behavior. The direct-state variant treats changes to selector state as its local learning operation. Its law, written for a selector S , is

$$S' = S \oplus (C \wedge U), \quad S, C, U \in \{0, 1\}. \quad (1)$$

In a supervised output comparison, $C = \text{OUT} \oplus \text{TARGET}$; U records whether a selector participated in computing that output. The state toggles exactly when $C = U = 1$; if either bit is zero, it stays the same. In this direct-state formulation, S is the selector state itself. The greedy variant defined later has a separate parameter m and an accept-or-revert test. Its update is not Eq. (1).

We first formalize the direct-state supervised per-output core: a forward pass supplies OUT, a target supplies TARGET, and a backward trace supplies USED. To define a runnable model beyond one comparison, we state a particular schedule for several examples and outputs, a causal count route, and a partial-prefix extension. We then describe the separately implemented greedy model and measure both on matched held-out splits. The direct-state routing choices are not deductions from the local law; changing their order or storage convention can change the computation.

2 A graph model with direct selectors

Let $G = (V, E)$ be an acyclic graph of input nodes, optional constant nodes, and multiplexer nodes. Each multiplexer v has data predecessors $a(v)$ and $b(v)$. Its output for input $x \in \{0, 1\}^p$ is

$$z_v(x; S) = \text{MUX}(z_{a(v)}(x; S), z_{b(v)}(x; S), \sigma_v(x; S)), \quad \text{MUX}(A, B, \sigma) = (\neg\sigma \wedge A) \vee (\sigma \wedge B). \quad (2)$$

The symbol σ_v denotes the value supplied to the selector input of node v . The basic graph permits two explicitly distinguished kinds of selector:

$$\sigma_v(x; S) = S_v \quad \text{for an adaptive node } v, \quad (3)$$

$$\sigma_v(x; S) = z_{d(v)}(x; S) \quad \text{for a signal-controlled node } v. \quad (4)$$

An adaptive node reads its own mutable selector state $S_v \in \{0, 1\}$ *directly*. A signal-controlled node reads an input or earlier graph signal through $d(v)$; it needs no separate inversion parameter. The mutable vector S consists of the adaptive selector states. An input-dependent selector can therefore be present without representing its learning as an XOR mask on top of it. This minimal distinction leaves open richer ways to make a selector context dependent.

The graph may share predecessors, making it a directed acyclic graph rather than necessarily a tree. Composing MUXes in multiple layers gives a larger network; a tree of seven 2:1 MUXes also implements an 8:1 choice. A set of q designated outputs defines $f_S : \{0, 1\}^p \rightarrow \{0, 1\}^q$. For a supervised input x_i and output bit j , let $C_{i,j}$ be its CHANGE bit and $U_{i,j,v}$ the USED bit of an adaptive node v , defined below. Its per-output update is

$$S_v^{(t+1)} = S_v^{(t)} \oplus (C_{i,j}^{(t)} \wedge U_{i,j,v}^{(t)}). \quad (5)$$

The indices identify the example, output bit, and direct selector state; they do not introduce a flip-mask variable. For this output comparison a change occurs only when $C_{i,j} = U_{i,j,v} = 1$. Once the state changes, the branch selected by that adaptive node may differ on other inputs as well. If the intended model instead uses context-specific states, their indexing and persistence require an additional definition.

2.1 Library realization

The library's `bfl::Graph` class uses this DAG model directly. `add_adaptive_mux` creates a MUX with a direct binary selector state, while `add_signal_mux` creates one whose selector is an earlier signal. Constants are dedicated nodes. This class trains adaptive states by Eq. (5); its routing convention and prefix parameter are defined below. The library's `bfl::GreedyModel` class, described later, implements a different learning rule.

3 Active paths and USED

For input x_i and output bit j , define the active dependency subgraph $D_j(x_i; S)$ recursively from output j . For each reached MUX, include the dependency of its selector, if it has one, and include only the data predecessor chosen by σ_v . A signal-controlled selector in Eq. (4) contributes its predecessor $d(v)$; an adaptive selector in Eq. (3) reads S_v directly. A shared node is counted once. Define

$$U_{i,j,v}(S) = \mathbf{1}\{v \in D_j(x_i; S)\}. \quad (6)$$

The USED bits can be computed explicitly from a forward pass followed by reverse traversal. For a fixed (i, j) , initialize $U_{i,j,v} = 0$ everywhere and set the designated output node's bit to one. Process marked MUXes in reverse topological order. If v is marked, send its mark to the selector predecessor when one exists, send it to $a(v)$ only when $\sigma_v = 0$, and send it to $b(v)$ only when $\sigma_v = 1$. Merge marks arriving along shared paths with OR. Algebraically, each marked MUX contributes

$$U_{i,j,a(v)} \leftarrow U_{i,j,a(v)} \vee (U_{i,j,v} \wedge \neg \sigma_v), \quad (7)$$

$$U_{i,j,b(v)} \leftarrow U_{i,j,b(v)} \vee (U_{i,j,v} \wedge \sigma_v), \quad (8)$$

$$U_{i,j,d(v)} \leftarrow U_{i,j,d(v)} \vee U_{i,j,v} \quad \text{if } v \text{ has a selector predecessor.} \quad (9)$$

On an 8:1 tree whose address bits come directly from inputs, the data trace visits one MUX at each of its three levels; a selector computed by another MUX also brings that selector’s dependencies into USED. This is a causal-path mark, independent of whether an implementation eagerly evaluates inactive branches. It is the U gate in Eq. (1), but it does not by itself say that changing S_v improves the output.

Proposition 1 (Inactive adaptive state). *For an adaptive node v , if $U_{i,j,v}(S) = 0$, changing only S_v leaves $f_S(x_i)_j$ unchanged.*

Proof. The value of output j depends only on its active selector and chosen data dependencies. Since v is absent from that dependency graph, changing its state cannot change a dependency already used to compute the output. Induction along the acyclic order leaves each relevant value, and hence the output, unchanged. $\square$

The converse need not hold: a used node may choose branches with equal values, or another part of the graph may cancel its effect. USED is therefore a way to restrict possible changes, not a complete credit-assignment mechanism.

4 Computing CHANGE and updating selectors

For supervised examples $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$, with $y_i \in \{0, 1\}^q$, compute the output $o_i = f_S(x_i)$. The CHANGE bit for output j is its binary mismatch:

$$C_{i,j} = o_{i,j} \oplus y_{i,j} = \text{OUT}_{i,j} \oplus \text{TARGET}_{i,j}. \quad (10)$$

Thus $C_{i,j} = 0$ for a correct bit and $C_{i,j} = 1$ for a wrong bit. For one (i, j) , compute all $U_{i,j,v}$ from the same pre-update forward pass, then update every adaptive selector on that output’s traced graph from the same state snapshot using Eq. (5). Equivalently, $\Delta S_v = C_{i,j} \wedge U_{i,j,v}$ and $S'_v = S_v \oplus \Delta S_v$ for every adaptive v . A correct output changes none of these selectors. An incorrect output changes only adaptive selectors on its USED path. No extra inversion mask F is part of this rule.

This describes a direct per-output update: after computing C and U , the corresponding selector states change without first computing a scalar loss or testing alternative parameter settings. Several errors still require a scheduling convention. The following choice makes repeated presentations unambiguous while retaining the original one-comparison law.

4.1 An explicit schedule for several examples and outputs

Fix an order of the examples $i = 1, \dots, n$ and output bits $j = 1, \dots, q$. Number their lexicographic comparisons by $k = (i - 1)q + j$, and let $S^{[k]}$ be the whole selector vector before comparison k . Compute a *fresh* forward output and backward trace at $S^{[k]}$, then simultaneously update all adaptive nodes for that comparison:

$$\begin{aligned} C^{[k]} &= f_{S^{[k]}}(x_i)_j \oplus y_{i,j}, \\ U_v^{[k]} &= U_{i,j,v}(S^{[k]}), \\ S_v^{[k+1]} &= S_v^{[k]} \oplus (C^{[k]} \wedge U_v^{[k]}) \quad \text{for every adaptive } v. \end{aligned} \quad (11)$$

One pass over all nq comparisons is an epoch; another epoch starts from its final state. No cross-example error is summed, voted on, or converted to a scalar objective. In particular, all traces for a single comparison come from its pre-update snapshot, whereas the next comparison sees the updated graph. The schedule is a *proposed completion*: the core equation alone neither selects an example order nor equates this online process with a batch update. Permuting examples or output bits can change the state trajectory.

Even perfect single-example correction does not imply convergence across examples. Take the one-node graph $f_S(x) = \text{MUX}(0, 1, S) = S$ and two presentations whose targets are respectively zero and one. Starting at $S = 0$, the first comparison leaves $S = 0$ and the second changes it to $S = 1$. On the next epoch the first changes it back to zero and the second changes it to one. Thus the epoch-end state remains one while the first target is repeatedly wrong. No selector state can satisfy both targets in this graph. This is an explicit limit of the online convention, not a failure of MUX representational completeness with a larger graph.

5 Examples and expressivity

The simplest direct-state example is $z_v = \text{MUX}(0, 1, S_v) = S_v$. For a single target $y = 1$ with $S_v = 0$, the forward output is zero, so $C = 0 \oplus 1 = 1$. The output node is USED, so $U = 1$ and Eq. (5) sets $S'_v = 0 \oplus (1 \wedge 1) = 1$. If that node were not on the active route to the chosen output, $U = 0$ would prevent its update. There is no dynamic selector signal and no additional inversion bit.

Signal-controlled MUXes can construct input-dependent functions. For example, if $\neg x_2$ is available from a Boolean subgraph, then

$$\text{MUX}(x_2, \neg x_2, x_1) = x_1 \oplus x_2. \quad (12)$$

This example explains the expressive role of an input-driven selector. Equation (12) contains no adaptive state, so it is not by itself an example of learning XOR with CHANGE. A trainable state could select between two already constructed functions, but learning the required subgraphs or topology is a separate problem.

Proposition 2 (Representational completeness of the MUX basis). *With input-controlled selectors, constants, and a graph whose size may grow with the target, any finite Boolean function $g : \{0, 1\}^p \rightarrow \{0, 1\}$ has a multiplexer representation.*

Proof. Shannon decomposition gives $g = \text{MUX}(g|_{x_k=0}, g|_{x_k=1}, x_k)$. Recursively decompose each cofactor until the leaves are constants. The resulting finite graph consists of signal-controlled MUXes and may share identical subgraphs. $\square$

This is a property of the computational basis, independent of the CHANGE law. It does not establish that a fixed graph can express every target or that direct-selector updates discover a useful representation. For a fixed graph with N adaptive state bits, at most 2^N distinct parameter settings exist, possibly representing fewer than 2^N distinct functions.

Indeed, representability does not even guarantee success on one repeated target for the specified update. Consider two adaptive states S_1, S_2 and the graph

$$z_2 = \text{MUX}(0, 1, S_2), \quad w = \text{MUX}(1, 0, z_2), \quad o = \text{MUX}(z_2, w, S_1) = S_1 \oplus S_2. \quad (13)$$

The first two MUXes supply S_2 and its complement; w has a signal-controlled selector. Both adaptive nodes are USED regardless of the current states: the root reads S_1 , while its chosen branch depends on z_2 . For target $o = 1$, initialization $(S_1, S_2) = (0, 0)$ gives $C = 1$ and toggles both states to $(1, 1)$, leaving $o = 0$. Repeating the presentation toggles both back. The graph represents the target at $(0, 1)$ or $(1, 0)$, yet this trajectory never reaches it. This example proves only a failure of this update from this initialization; it does not exclude other graphs or update schedules.

6 Count routing from effective prior changes

Mathematical definition. Order graph nodes topologically, and let $v_0, \dots, v_{m-1}$ be the MUXes in that order. For a supervised comparison k , define the event at MUX v_i by

$$A_i^{[k]} = C_i^{[k]} \wedge U_i^{[k]}, \quad C_i^{[k]} = \begin{cases} C_i^{[k]}, & v_i \text{ adaptive,} \\ 0, & v_i \text{ signal-controlled.} \end{cases} \quad (14)$$

A signal-controlled MUX has no mutable selector state, so it cannot produce a state modification. For adaptive MUXes, $A_i = 1$ precisely when the output is wrong and that MUX is USED. The route for MUX v_j is

$$N_j^{[k]} = \sum_{i < j} A_i^{[k]}, \quad R_j^{[k]} = \text{bin}(N_j^{[k]}). \quad (15)$$

Here N_j is a nonnegative integer and bin is its ordinary base-two representation $(0, 1, 10, 11, \dots)$. It is not a bitmask of earlier MUX positions. The current MUX’s own event does not enter its route. Equivalently, traverse MUXes in topological order, assign $R_j \leftarrow \text{bin}(N)$, then set $N \leftarrow N + A_j$, starting from $N = 0$. The library stores the integer N_j as the route key; `binary_route` exposes its binary digits. For example, preceding event patterns 1010 and 0101 both give $N_j = 2$ and $R_j = 10_2$, whereas 1101 gives $N_j = 3$ and $R_j = 11_2$. At the same MUX, equal counts select the same state bank. This is structural reuse; equality of routes does not assert equality of external inputs X , targets Y , or active graph paths. A route describes prior internal processing.

Causal timing and update. `CHANGE` depends on a target and `USED` depends on the forward-selected paths, so the event vector for a comparison is only known after its forward and backward passes. Let $A^{[k-1]}$ be the events recorded from the previous comparison; initialize it to all zero. Comparison k evaluates each MUX with $N_j^{[k-1]}$ from Eq. (15). An adaptive MUX reads the state $S_{v_j}^{(N_j^{[k-1]})}$ for that route. After computing $C^{[k]}$ and all $U^{[k]}$ from this same snapshot, it updates each addressed state exactly by

$$S_{v_j}^{(N_j^{[k-1]})'} = S_{v_j}^{(N_j^{[k-1]})} \oplus (C^{[k]} \wedge U_{v_j}^{[k]}), \quad (16)$$

and records $A^{[k]}$ for the next comparison. Routing chooses which direct state is read; it never substitutes for Eq. (1). New route banks start at the MUX’s initial state and are materialized only when changed. The graph’s route is computed in one topological pass by adding each preceding event after assigning the current MUX’s route. No overflow occurs while the count is bounded by the number of graph MUXes.

Training traverses examples and outputs in a fixed order for each epoch, with a fresh forward pass for each comparison. Inference has no target from which to derive a new `CHANGE` bit. It therefore reads the last recorded event vector without modifying it; callers may explicitly restore a valid event vector with `set_routing_events`. This is a model-context convention, not an input-conditioned router. A model requires its graph, route-specific states, and final event vector to reproduce predictions. The library currently has no persistence format; copying a `Graph` preserves these values in memory.

Hypothesis. Sharing a state when counts coincide may exert compression or regularization pressure, because several event patterns update the same route bank. This is a hypothesis about learning behavior, not a consequence of the route equation. Distinct patterns can also interfere at a shared state, and the online rule can oscillate. Held-out measurements are required before any generalization claim.

7 Partial-prefix route experiment

The count route deliberately merges different effective-event positions. To retain a bounded amount of positional information, let $k \in \{0, \dots, 64\}$ and define, for MUX v_j ,

$$P_{j,k} = \sum_{i=0}^{\min(j,k)-1} A_i 2^{\min(j,k)-1-i}, \quad R_{j,k} = (N_j, P_{j,k}). \quad (17)$$

The bits of $P_{j,k}$ are the first k MUX event bits that precede v_j , in topological graph order; signal-controlled MUXes contribute zero bits. At a fixed node their prefix length is fixed. With $k = 0$, $P_{j,0} = 0$ and the route is exactly the count route. For $k = 2$, preceding patterns 1010 and 0101 have the same count two but prefixes 10_2 and 01_2 , so they address different states. Patterns 1010 and 1001 still share a route. The implementation stores the pair as a `RouteKey`; `Graph(k)` selects k , and the default constructor uses $k = 0$. The causal timing, USED trace, and update law remain those of Eq. (16), with $R_{j,k}$ replacing the count address. This is a deliberately partial bitmask: events beyond the first k positions contribute to the count but not to the prefix.

8 Measured benchmark result

Experimental result. The reproducible C++ benchmark enumerates the binary domains of width 4, 6, 8, shuffles them with seeds 1, 2, 3, uses the first half for online training and the next quarter as disjoint held-out inputs, and measures parity and majority output bits. BFL uses two full lookup trees: adaptive leaves select between constants, and signal-controlled internal MUXes select by input bits. We compare $k = 0, 4, 8, 16$ with identical splits, initialization, and update order, for one and three epochs. The majority baseline stores the most frequent training bit per output. Table 1 gives one-epoch arithmetic means from `results/run_2026-09-30-prefix/benchmark.csv`; the CSV also includes per-seed timings, sizes, and other metrics. Distinct routes rise with k ,

bits	k	train acc.	test acc.	routes	USED addresses
4	0	.875	.375	2.00	16
4	4	.875	.375	4.67	16
4	8	.875	.375	8.67	16
4	16	.854	.375	16.67	16
6	0	.818	.510	2.00	64
6	4	.818	.510	3.67	64
6	8	.818	.510	7.67	64
6	16	.818	.510	15.67	64
8	0	.861	.448	2.00	256
8	4	.861	.448	3.67	256
8	8	.861	.448	6.33	256
8	16	.861	.448	14.33	256

Table 1: Mean bit accuracies, distinct route keys, and distinct adaptive selector addresses reached by USED traces across three seeds after one epoch.

confirming that the prefix separates event patterns that count routing merges. Yet the number of adaptive $(v, R_{j,k})$ addresses actually reached by USED is unchanged at every width. The lookup tree assigns each input its own adaptive leaf, so this architecture gives different training inputs little opportunity to compete for the same addressed state. After three epochs every BFL variant reached training bit accuracy 1.000 at all three widths; held-out accuracy remained .375, .510, and .448, respectively, for every k . Inference keeps the final event vector as a common context for all held-out inputs. These measurements show route diversity without a held-out gain and do not validate a generalization hypothesis for partial-prefix routing.

9 A separate greedy MUX model

The library also provides `bf1::GreedyModel`, a C++ implementation of a layered random MUX network. This model has a different selector parameterization and training rule from `bf1::Graph`. For node v , let $a(v)$ and $b(v)$ be data predecessors, $d(v)$ a selector predecessor, and $m_v \in \{0, 1\}$ a learned inversion bit. Each predecessor is an input column or a node in the immediately preceding layer. The node output is

$$z_v(x; m) = \text{MUX}(z_{a(v)}(x; m), z_{b(v)}(x; m), z_{d(v)}(x; m) \oplus m_v). \quad (18)$$

The topology stays fixed during one training run; training changes only m . A constant input column, when supplied, gives the network access to a fixed one. Each node’s three source indices and state bit are stored in one 64-bit word. Forward evaluation packs up to 64 examples into each machine word; this changes the execution layout, not Eq. (18).

For a training set $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$ with q output bits, define the global Hamming objective

$$L_{\mathcal{D}}(m) = \sum_{i=1}^n \sum_{j=1}^q (f_m(x_i)_j \oplus y_{i,j}). \quad (19)$$

For each currently wrong output bit, a USED trace marks its output MUX, the selector dependency, and the selected data dependency, recursively. The union of marked MUXes over the training set supplies candidate nodes. At the start of each epoch the candidates may be shuffled and capped. For each candidate v in sequence, the implementation proposes $m' = m \oplus e_v$, recomputes $L_{\mathcal{D}}(m')$ on the entire training set, and keeps the proposal only if

$$L_{\mathcal{D}}(m') < L_{\mathcal{D}}(m). \quad (20)$$

Otherwise it restores the old bit. A later candidate is scored against the state left by earlier accepted flips. Candidate traces are refreshed at the next epoch. This is a global, strictly improving, single-bit search; Eq. (1) is not its update law.

Proposition 3 (Monotone training error for the greedy model). *Within one training run, the global Hamming error cannot increase after a candidate test. The number of accepted single-bit flips is at most the initial Hamming error.*

Proof. An accepted flip decreases the nonnegative integer $L_{\mathcal{D}}$ by at least one; a rejected flip restores the preceding state. Thus every accepted flip consumes at least one unit of the initial error, and all other tests leave it unchanged. $\square$

This proposition concerns the training set only. Candidate limits, topology, and local minima can prevent zero training error; monotonic training error does not imply held-out improvement.

9.1 Matched generalization comparison

We compare this greedy model, a direct-state lookup `Graph`, and a one-hidden-layer neural network (MLP) on the same splits. Each example has eight varying input bits and a constant-one column. The targets are OR or XOR of the first two bits, or majority of the first three; the remaining varying bits are irrelevant to the label. For each of 20 seeds, the 256 distinct eight-bit inputs are shuffled into 128 training, 64 validation, and 64 held-out test examples. The test split is used only after training and topology selection.

The greedy model has three hidden layers of 64 MUXes and one output MUX. It runs at most 100 epochs, tests at most 128 candidate nodes per epoch, and stops after 15 epochs without an accepted flip. We report the first random topology and the best of 16 independently trained topologies, choosing the latter by lowest validation error and then lowest training error. The MLP has 16 tanh hidden units, one sigmoid-equivalent binary output, and 1500 full-batch

Adam epochs. The direct-state control has one adaptive lookup leaf per eight-bit input and runs three online epochs. Every model receives the same input columns and split. Table 2 gives arithmetic means over the 20 seeds from `results/run_2026-09-30-generalization/benchmark.csv`; `scripts/reproduce_generalization.sh` reproduces the comparison.

task	model	train acc.	validation acc.	test acc.
OR2	greedy, first topology	.797	.685	.668
OR2	greedy, selected of 16	.880	.825	.762
OR2	direct-state lookup	1.000	.245	.245
OR2	MLP	1.000	1.000	1.000
Majority3	greedy, first topology	.773	.670	.637
Majority3	greedy, selected of 16	.834	.793	.733
Majority3	direct-state lookup	1.000	.477	.503
Majority3	MLP	1.000	1.000	1.000
XOR2	greedy, first topology	.679	.557	.546
XOR2	greedy, selected of 16	.746	.697	.608
XOR2	direct-state lookup	1.000	.498	.488
XOR2	MLP	1.000	1.000	1.000

Table 2: Mean bit accuracies on matched training, validation, and held-out test splits. Topology selection uses validation data only.

On OR2, the selected greedy model reaches .762 held-out accuracy versus 1.000 for the MLP under this protocol. This new result is not a reproduction of the previously reported approximate 80% versus 100% comparison: its original dataset and split are unavailable. Selection improves the greedy model’s mean test score over its first topology on all three tasks, while its absolute accuracy depends on the task. The direct-state control memorizes observed input addresses, so its test score does not isolate the effect of Eq. (1). The models also differ in topology, training rule, and search budget; matched data splits alone do not establish an algorithm-wide ranking.

10 Relation to prior work and open questions

Boolean decision diagrams use shared graph representations of Boolean functions [1]. Cartesian genetic programming also studies computations represented as acyclic graphs [2]. Binary neural networks provide an empirical comparison point, although their state and training procedures differ [3]. These antecedents do not resolve whether BFL’s direct CHANGE law yields an effective learning rule.

The online schedule and count route remove operational ambiguities for the direct-state model. They do not determine a best example order, resolve conflicting updates, or select a useful graph topology. The greedy model’s improvement test resolves each proposed bit flip against training Hamming error, but leaves topology choice and held-out behavior empirical. The measurements above describe these specific implementations and tasks; broader claims require other datasets and controlled comparisons of graph families and search budgets.

11 Conclusion

The direct-state model computes $C = \text{OUT} \oplus \text{TARGET}$, traces $U = \text{USED}$, and updates a direct selector by $S' = S \oplus (C \wedge U)$. The tested partial-prefix route created more route keys in a lookup graph without improving held-out accuracy. The separate greedy model learns inversion bits on signal-controlled MUXes and accepts only flips that strictly reduce global training Hamming error. With validation-based selection among 16 topologies, it reached .762, .733, and .608 mean held-out accuracy on OR2, Majority3, and XOR2; the matched MLP reached 1.000 on each. Both models are implemented in C++; their update laws and topologies must be evaluated separately. Neither MUX expressivity nor monotone greedy training error proves generalization.

References

- [1] R. E. Bryant, “Graph-Based Algorithms for Boolean Function Manipulation,” *IEEE Transactions on Computers*, C-35(8), pp. 677–691, 1986. <https://doi.org/10.1109/TC.1986.1676819>.
- [2] J. F. Miller and P. Thomson, “Cartesian Genetic Programming,” *European Conference on Genetic Programming*, LNCS 1802, pp. 121–132, 2000. https://doi.org/10.1007/978-3-540-46239-2_9.
- [3] M. Courbariaux, I. Hubara, D. Soudry, R. El-Yaniv, and Y. Bengio, “Binarized Neural Networks: Training Deep Neural Networks with Weights and Activations Constrained to +1 or -1,” 2016. <https://arxiv.org/abs/1602.02830>.